

Group030_A1.zip

这是我们的具体数据，请你帮我给我 validation 应该按照什么分类，参照 schema、types、missing、PK/FK、source/row flow、overlap...给出我完整的分析建议

思考了 2m 16s

可以。根据你们 **Group030_A1.zip** 的真实数据结构，我建议不要把 validation 简单写成 “schema / missing / duplicate” 三四项，而是整理成一个完整的 **8 类 validation framework**。这样既覆盖老师明显要求的 schema、types、missing、PK/FK、source/row flow、overlap，又能体现你们确实理解了这个 assignment 的核心：**JSON + XML 多源整合、去重、reconciliation、规范化和业务一致性验证**。

我已经实际检查了你们的数据，下面这些数字可以直接作为你们后续 validation 的基准。

1. Validation 总体分类

我建议最终按这 8 类组织：

Validation category	主要检查什么	优先级
1. Schema validation	表、字段、顺序、grain	必须
2. Type & format validation	string/date/datetime/number/boolean 格式	必须
3. Missing / null validation	nullable 与 non-nullable	必须
4. PK / uniqueness validation	主键、重复记录、grain	必须
5. FK / referential integrity	六张表之间关系	必须
6. Source / row-flow reconciliation	raw → dedup → overlap → final	非常重要
7. Cross-source overlap validation	JSON/XML 重叠实体是否正确合并	非常重要

Validation category	主要检查什么	优先级
8. Business / semantic validation	金额、日期、文本、regex、业务逻辑	HD 加分重点

2. Schema validation

你们最终应该严格生成 6 张表：

1. orders
2. order_items
3. customers
4. deliveries
5. products
6. product_reviews

必须以 public_data_dictionary.csv 为唯一 schema contract。

例如：

orders

必须：

- 23 columns
- one row per order
- 字段名完全一致
- 顺序完全一致

例如：

order_id

source_system_record_id

customer_id

order_timestamp

...

customer_note_clean

promo_code

Validation 建议:

```
assert list(orders.columns) == expected_orders_columns
```

```
assert len(orders.columns) == 23
```

所有六张表都做一次。

Report 可以写

Schema validation confirmed that all six output tables matched the published data dictionary in table grain, column names, column order and required field count.

这个属于 structural validation。

3. Types / Format validation

这一部分你们的数据尤其重要，因为 JSON 和 XML 原始格式不一样。

JSON

JSON 里很多已经是比较标准的:

2018-12-09 13:00:00

4872.08

True

XML

XML 是另一套格式:

25/04/2018 09:33:00

AUD 622.90

AUD 2,393.92

10%

Y / N

因此应该 validation:

Date / datetime

例如：

orders.order_timestamp → datetime

customers.signup_date → date

products.launch_date → date

deliveries.dispatch_date → date

reviews.review_timestamp → datetime

统一后验证：

```
assert pd.api.types.is_datetime64_any_dtype(...)
```

或者逐行：

```
pd.to_datetime(..., errors="raise")
```

Number

XML：

AUD 2,393.92

必须变：

2393.92

然后检查：

```
unit_price >= 0
```

```
order_price >= 0
```

```
delivery_cost >= 0
```

Boolean

不同来源：

True / False

Y / N

最终必须统一 boolean。

特别：

expedited_delivery

marketing_consent

on_time_in_full

signature_required

recyclable_packaging

active_flag

verified_purchase

contains_non_latin_script

Percentage

例如 XML:

10%

最终:

10

不是:

0.10

因为 dictionary 的 comparison rule 对 coupon_discount 是 exact。

4. Missing value validation

这个不要写成:

"we checked missing values."

要根据 data dictionary 的 nullable 明确检查。

你们 dictionary 中最明显允许 missing 的包括:

orders.coupon_code

orders.promo_code

其他绝大多数 output fields:

nullable = False

因此建议直接做:

for col in non_nullable_columns:

```
assert df[col].notna().all()
```

同时检查：

None

""

" "

"NULL"

"null"

"N/A"

全部规范化。

你们真实数据里的一个例子

我检查 JSON orders.header.couponCode：

原始 2,818 行中大约：

2,109 条是空字符串。

这是正常的，因为 coupon_code 本来就是 nullable。

因此：

""

不应该保留为 empty string，而应该转为规定的 missing 表示。

README 特别要求：

use the literal NaN for prescribed missing values.

所以这项必须专门 validation。

5. PK / Uniqueness validation

这是非常重要的一部分。

六张表分别建议如下。

Table	PK / uniqueness
orders	order_id
order_items	order_item_id
customers	customer_id
deliveries	delivery_id, 同时 order_id 应唯一
products	product_id
product_reviews	review_id

代码逻辑:

```
assert orders["order_id"].is_unique
```

```
assert order_items["order_item_id"].is_unique
```

```
assert customers["customer_id"].is_unique
```

```
assert deliveries["delivery_id"].is_unique
```

```
assert products["product_id"].is_unique
```

```
assert product_reviews["review_id"].is_unique
```

6. 这里是你们数据真正值得写进报告的发现

原始数据故意有重复记录。

Orders

commerce JSON:

Raw rows	2818
----------	------

Unique order_id	2750
-----------------	------

Duplicate rows	68
----------------	----

operations XML:

Raw rows 2818

Unique order_id 2750

Duplicate rows 68

所以你们不能写:

$2818 + 2818 = 5636$ orders

这是错的。

7. Cross-source overlap

这是这个作业很重要的点。

两个来源:

JSON unique orders = 2750

XML unique orders = 2750

但两个来源之间存在:

500 个共同 order_id

因此 canonical orders:

$$2750 + 2750 - 500 = 5000$$

你们最终 orders 的目标应是:

5,000 rows

前提是你们 reconciliation 逻辑正确。

8. Product reviews 也完全是类似设计

JSON:

raw reviews = 3946

unique review_id = 3850

duplicate rows = 96

XML:

raw reviews = 3946

unique review_id = 3850

duplicate rows = 96

跨源:

overlap review_id = 700

因此:

$$3850 + 3850 - 700 = 7000$$

最终:

product_reviews = 7,000 canonical reviews

这个数字建议一定写入 validation report。

9. Order items row flow

我也实际检查了 item。

JSON

Raw item rows 8823

Unique item IDs 8612

XML

Raw item rows 8885

Unique item IDs 8668

跨源共同 item:

1557

所以:

$$8612 + 8668 - 1557 = 15723$$

建议最终:

order_items $\approx$ 15,723 canonical rows

严格来说应该以最终 implementation 的 stable-key reconciliation 结果再次 assert。

10. Customers / products

这两张更简单。

customers

来自 commerce JSON:

500 raw

500 unique customer_id

所以:

customers = 500

products

来自 Operations XML:

1000 raw

1000 unique product_id

所以:

products = 1,000

11. Deliveries

你们当前所有 raw orders 都:

order_status = Completed

delivery_status = Delivered

而 dictionary 定义:

one row per completed order

因此 canonical orders 是 5,000 时:

deliveries 应该也是 **5,000 rows**

而且:

```
assert deliveries["order_id"].is_unique

assert set(deliveries.order_id) == set(orders.order_id)
```

这个 validation 非常漂亮。

12. 推荐你们最终做一张 Row Flow 表

这个非常适合 HD report。

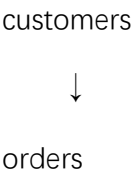
Entity	JSON raw	JSON unique	XML raw	XML unique	Cross-source overlap	Final expected
Orders	2,818	2,750	2,818	2,750	500	5,000
Order items	8,823	8,612	8,885	8,668	1,557	15,723
Reviews	3,946	3,850	3,946	3,850	700	7,000
Customers	500	500	—	—	—	500
Products	—	—	1,000	1,000	—	1,000
Deliveries	2,818	2,750	2,818	2,750	500	5,000

这张表我认为是你们 validation 最值得展示的一张。

13. FK / Referential Integrity

这个一定不要漏。

你们实际上是一个典型关系模型：





order_items → products



product_reviews

orders → deliveries

必须检查以下 FK

orders → customers

orders.customer_id

→ customers.customer_id

assert set(orders.customer_id) <= set(customers.customer_id)

order_items → orders

order_items.order_id

→ orders.order_id

order_items → products

order_items.product_id

→ products.product_id

deliveries → orders

deliveries.order_id

→ orders.order_id

并且应该 1:1:

assert deliveries.order_id.is_unique

reviews → orders

product_reviews.order_id

→ orders.order_id

reviews → order_items

product_reviews.order_item_id

→ order_items.order_item_id

reviews → products

product_reviews.product_id

→ products.product_id

reviews → customers

product_reviews.customer_id

→ customers.customer_id

我检查原始 canonical key union 后

目前这些关系表现很好:

Order → customer missing FK = 0

Item → product missing FK = 0

Item → order missing FK = 0

Review → order/item/product/customer = 0 missing

说明原始数据关系设计是完整的。

但你们还是必须 validation, 因为 ETL 过程中很容易把 FK 搞丢。

14. 不仅要检查 FK 存在, 还要检查 cross-key consistency

这是更高一级的 validation。

例如一条 review:

review.order_id = HORD000866

review.order_item_id = HITM0002752

review.product_id = PRD0560

不能只检查这三个 ID 各自存在。

还要检查：

HITM0002752 确实属于 HORD000866

而且：

HITM0002752 对应的 product_id 确实是 PRD0560

所以：

review

merge order_items on order_item_id

然后：

assert review.order_id == item.order_id

assert review.product_id == item.product_id

这叫 semantic FK consistency。

这个写进去很加分。

15. Source overlap validation 应该怎么做

这里建议单独成为一个 section：

Step 1 Source-internal duplicate

例如 JSON：

HORDxxxx

HORDxxxx

相同 ID 出现两次。

检查：

same key + same values

还是：

same key + conflicting values

必须区分：

Exact duplicate

可以安全删除。

Conflicting duplicate

不能无脑 `drop_duplicates()`。

需要：

flag → compare → deterministic resolution rule

你们这份数据源内的重复绝大多数是 exact duplicate。

因此报告可以描述：

Duplicate identifiers were grouped before deduplication and records sharing a stable key were compared field-by-field to distinguish exact duplication from conflicting duplication.

这个比：

`drop_duplicates()`

好很多。

16. Cross-source overlap

例如：

JSON HORD001234

XML HORD001234

不要变成两条。

流程应该：

JSON record



normalise



stable order_id



compare

↓
XML record
↓
canonical record

Validation:

JSON_only

XML_only

Both

例如 orders:

JSON only = 2250

XML only = 2250

Both = 500

最终:

$2250 + 2250 + 500 = 5000$

建议做一个简单柱状图或 Venn-style 表达:

JSON only	2250
-----------	------

Both	500
------	-----

XML only	2250
----------	------

Reviews:

JSON only	3150
-----------	------

Both	700
------	-----

XML only	3150
----------	------

这会非常直观。

17. Arithmetic / business validation

这部分是 HD 与普通 validation 很容易拉开差距的地方。

A. line revenue

必须:

$$line_revenue = quantity \times unit_price$$

允许:

tolerance = 0.01

代码:

```
abs(  
    line_revenue -  
    quantity * unit_price  
) <= 0.01
```

我检查 JSON unique orders, 目前这个关系是成立的。

18. order_price

应验证:

$$order_price = \sum line_revenue$$

按 order_id group:

```
item_sum = (  
    order_items  
    .groupby("order_id")["line_revenue"]  
    .sum()  
)
```

然后和:

orders.order_price

比较。

Tolerance:

0.01

我检查 JSON 部分也是一致的。

19. order_total

根据你们实际数据，重要的一点是：

不要把 tax_amount 再加一次。

例如真实记录：

order_price = 4872.08

coupon_discount = 5

delivery_charges = 19.09

order_total = 4647.57

计算：

$$4872.08 \times 0.95 + 19.09 = 4647.566$$

round 后：

4647.57

所以你们应该验证类似：

$$order_total = order_price \times (1 - coupon_discount/100) + delivery_charges$$

而 tax_amount 更像 GST component，不是再额外加到 total 上。

这个地方很值得明确写出来。

20. Tax validation

澳洲 GST 数据看起来还可以进一步验证：

例如：

$$4872.08 / 11 \approx 442.92$$

恰好就是：

$$tax_amount = 442.92$$

所以你们可以检查是否：

$$tax_amount \approx order_price/11$$

至少对于：

GST_STANDARD

或适用类别。

但这一项建议作为 semantic validation, 不要当 schema requirement。

21. Delivery validation

非常推荐检查：

日期顺序

概念上：

order_date

<= dispatch_date

<= delivered_date

但注意：

order_timestamp 有小时分钟，而 dispatch 只有 date。

所以比较时最好：

order_timestamp.date()

不要拿：

2018-01-20 18:00

直接和：

2018-01-20 00:00

比较，否则会制造假异常。

Promised / delivered

检查：

delay_days

应该和：

$$\max(0, delivered_date - promised_date)$$

逻辑一致。

同时：

on_time_in_full = True

通常应该：

delivered_date <= promised_date

反之 delayed。

22. Temporal validation

建议形成完整的一组：

Customer

signup_date <= order_timestamp

Product

launch_date <= order_timestamp

Order/Delivery

order_date <= dispatch_date

dispatch_date <= delivered_date

Review

至少：

review_timestamp >= order_timestamp

最好进一步：

review_date >= delivered_date

如果业务设定为收货后才能 review。

23. Domain / range validation

也很容易做，但报告看起来会很完整。

Rating

$1 \leq \text{rating} \leq 5$

Quantity

$\text{quantity} \geq 1$

$\text{quantity} \% 1 == 0$

Discount

$0 \leq \text{coupon_discount} \leq 100$

Latitude

$-90 \leq \text{customer_lat} \leq 90$

Longitude

$-180 \leq \text{customer_long} \leq 180$

更进一步，因为数据明显在 Melbourne：

可以检查地理范围是否合理，但不要过度 hard-code。

Price

$\text{unit_price} > 0$

$\text{unit_cost} \geq 0$

$\text{order_price} \geq 0$

$\text{delivery_charges} \geq 0$

$\text{delivery_cost} \geq 0$

Review votes

$\text{helpful_votes} \geq 0$

24. Category / enum validation

建议把 categorical values 做 allowed-set validation。

例如：

order_status

sales_channel

payment_method

season

device_type

carrier

service_level

delivery_status

loyalty_tier

customer_segment

language_code

writing_style

delivery_experience

value_experience

方法：

```
unexpected = set(df[col]) - allowed_values
```

```
assert not unexpected
```

或者在 report 里展示 distinct counts。

25. Text cleaning validation

README 对 text cleaning 要求非常明确，因此一定单独验证。

涉及：

customer_note_clean

product_description_clean

review_body_clean

review_body_latin_analysis

要求去掉：

HTML tags

[SYSTEM]

[CATALOGUE]

[VERIFIED_PURCHASE]

URLs

emoji

repeated whitespace

因此你们应该反向 validation:

assert not contains_html

assert not contains_url

assert not contains_forbidden_marker

assert not contains_emoji

assert not contains_repeated_whitespace

这个比肉眼检查强很多。

26. Multilingual review 特别注意

README 明确要求:

Preserve the lower-case multilingual review in review_body_clean.

所以:

review_body_clean

不能粗暴:

re.sub("[^a-zA-Z]", "", ...)

否则中文、日文等全部丢掉。

另外:

review_body_latin_analysis

才是只保留 Latin-script letters 的版本。

而且还必须保留欧洲变音字符，例如：

é

ö

ü

ñ

ç

不能只：

[a-zA-Z]

27. Regex extraction validation

README 已经直接告诉你们 pattern。

Order reference

[HC]ORD\d{6}

SKU

类似：

SKU-[A-Za-z0-9]+

Promo

B[1-5]SAVE-\d{2}

要产生：

extracted_order_reference

extracted_product_sku

promo_code

但 validation 不只是：

成功运行 regex。

而应该检查 extraction consistency。

例如：

review.extracted_order_reference

==

review.order_id

如果评论正文存在 reference。

同样：

extracted_product_sku

应该能 join：

products.product_sku

进一步检查它对应的：

product_id

与 review.product_id 一致。

这个 validation 很强。

28. Derived text fields

例如：

review_length_chars

review_word_count

contains_non_latin_script

不要直接相信 ETL 结果。

重新计算后验证：

char count

review_length_chars == len(review_body_clean)

word count

按照你们定义的 tokenisation：

review_word_count == ...

non-Latin

如果 cleaned review 有非 Latin script：

contains_non_latin_script = True

否则 false。

29. Warehouse validation

这个是我建议你们增加的一项，很适合作为 advanced validation。

XML 里还有：

WarehouseDirectory

有：

Bakers

Thompson

...

以及 latitude / longitude。

orders 又有：

customer_lat

customer_long

nearest_warehouse

所以可以用 Haversine distance 重新计算：

customer coordinates

→ distance to each warehouse

→ minimum distance warehouse

然后：

calculated_nearest_warehouse

==

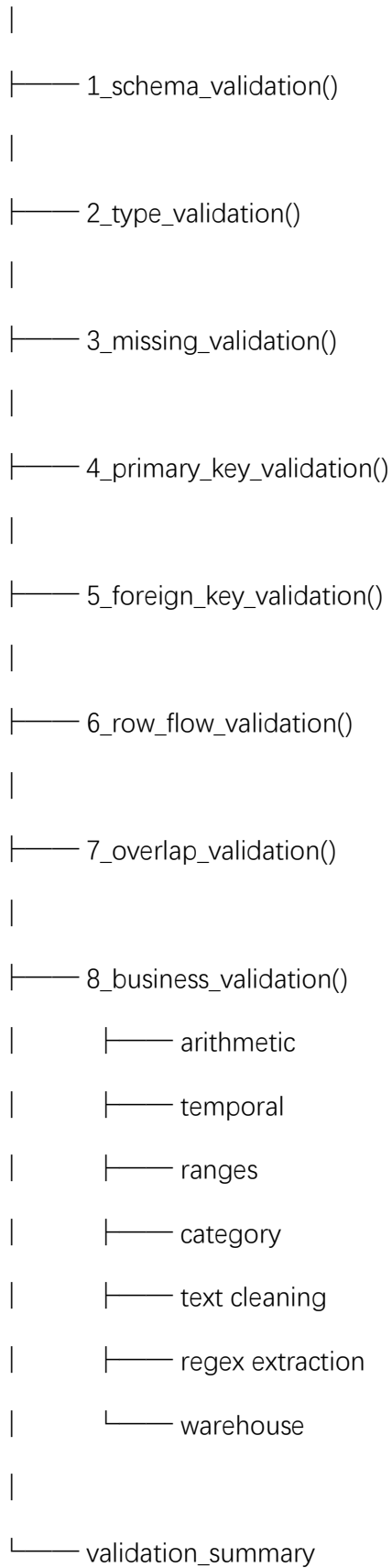
orders.nearest_warehouse

这是非常漂亮的 semantic validation。

30. 我建议你们最终 validation 代码按这个结构

不要散着写。

VALIDATION



最终统一输出：

PASS / FAIL

而不是中途一堆 print。

31. 最终 Validation Summary 最好长这样

Check	Result	Violations
Schema conformity	PASS	0
Data type conformity	PASS	0
Non-null constraints	PASS	0
PK uniqueness	PASS	0
FK integrity	PASS	0
Order source reconciliation	PASS	0
Review source reconciliation	PASS	0
Item source reconciliation	PASS	0
Line revenue arithmetic	PASS	0
Order price reconciliation	PASS	0
Order total reconciliation	PASS	0
Delivery temporal consistency	PASS	0
Review temporal consistency	PASS	0

Check	Result	Violations
Regex extraction	PASS	0
Text-cleaning residual check	PASS	0

这张表特别适合作为 report 的 evidence。

32. 如果按 HD 标准，我认为最重要的是这五层

你们不要平均用力。

优先把下面做好：

第一层：Schema

最终六张表严格符合 data dictionary。

第二层：Integrity

PK + FK + one-to-one / one-to-many grain 正确。

第三层：Row flow

明确解释：

2818 → 2750

2818 → 2750

2750 + 2750 - 500 → 5000

这个一定要有。

第四层：Overlap / reconciliation

不是简单 concat，而是：

source dedup

→ stable key

→ cross-source matching

→ compare

→ canonical record

第五层：Semantic validation

至少做：

line arithmetic

order arithmetic

temporal logic

regex/text cleaning

你们这份数据目前最核心的 **expected row counts**

建议你们直接拿这几个数和最终代码结果对：

customers 500

products 1000

orders 5000

deliveries 5000

order_items 15723

product_reviews 7000

其中最关键的 source reconciliation 是：

Orders:

JSON raw 2818

→ unique 2750

XML raw 2818

→ unique 2750

overlap 500

→ final 5000

以及：

Reviews:

JSON raw	3946
----------	------

→ unique	3850
----------	------

XML raw	3946
---------	------

→ unique	3850
----------	------

overlap	700
---------	-----

→ final	7000
---------	------

这两组 row-flow 我强烈建议放进 report，而不是只藏在代码里。

如果你们 validation 部分按上面组织，我认为已经比单纯的 schema + missing + duplicate + FK 完整很多，也更符合这个数据集真正思考的 **multi-source reconciliation** 和 **data integrity**。